

```

# ==== core/common.py ====
def empirical_hbase(rows, min_risk=30):
    r = rows[rows["weeks_obs"] >= rows["step_k"]]
    g = r.groupby(["app_id", "step_k"]).agg(surv=("survived", "sum"),
        n=("survived", "size")).reset_index()
    g = g[g["n"] >= min_risk]
    med_surv = g.groupby("step_k").apply(lambda d: (d["surv"] /
        d["n"]).median()).reindex(range(1, HMAX + 1))
    return (1 - med_surv)

# ==== web/v3/ltv_v3/map_web.py ====
import numpy as np
import pandas as pd

from core.common import HMAX, K_SHRINK, empirical_hbase

from ltv_v3.config import (
    MAP_LEVERS_WEB, MAP_MIN_ROWS_WEB, MAP_CLIP, CLEAN_STEP_MAX, SANITY_TOL,
    H_EXT,
)

CURVE_MAX = H_EXT

MIN_COHORTS_FOR_HBASE = 3

def _prep(mx: pd.DataFrame) -> pd.DataFrame:
    mx = mx.copy()
    mx["billday_bin"] = pd.cut(
        mx["billing_day_of_month"], bins=[0, 10, 20, 31],
        labels=["d01_10", "d11_20", "d21_31"],
    ).astype(str)
    for col in ("geo", "media_source", "funnel"):
        if col in mx.columns:
            mx[col] = mx[col].astype("string").fillna("(none)").replace("",
                "(none)")

    if "utm_source" not in mx.columns and "media_source" in mx.columns:
        mx["utm_source"] = mx["media_source"]
    return mx

def _collapse_small(df: pd.DataFrame, small_categories: dict) -> pd.DataFrame:
    df = df.copy()
    for lever, small in small_categories.items():
        if lever in df.columns:
            df[lever] = df[lever].where(~df[lever].isin(small), "other")
    return df

def build_h_base(train_rows: pd.DataFrame, ios_h_base: pd.Series = None) ->
tuple:
    at_risk = train_rows[train_rows["weeks_obs"] >= train_rows["step_k"]]
    support = (
        at_risk.groupby(["app_id", "step_k"]).size().rename("n").reset_index()
        .query("n >= 30").groupby("step_k")["app_id"].nunique()
    )
    trusted = [k for k in range(1, HMAX + 1) if support.get(k, 0) >=
        MIN_COHORTS_FOR_HBASE]
    if not trusted:
        raise RuntimeError("No step has enough cohorts to estimate a web
            h_base")
    k_max_web = max(trusted)

    web_h = empirical_hbase(train_rows).reindex(range(1, CURVE_MAX + 1))
    h = web_h.copy()

    if ios_h_base is not None:
        ios = ios_h_base.reindex(range(1, CURVE_MAX + 1))
        seam = web_h.loc[k_max_web] / ios.loc[k_max_web] if
            ios.loc[k_max_web] > 0 else 1.0
        for k in range(k_max_web + 1, CURVE_MAX + 1):

```

```

        h.loc[k] = ios.loc[k] * seam
    else:
        h.loc[k_max_web + 1:] = web_h.loc[k_max_web]

    h = h.ffill().bfill()
    return h, k_max_web

def fit(mx: pd.DataFrame, ios_h_base: pd.Series = None, crash_cohorts: set =
None,
        levers: list = None, min_rows: int = MAP_MIN_ROWS_WEB) -> dict:
    levers = list(levers) if levers is not None else list(MAP_LEVERS_WEB)
    crash_cohorts = crash_cohorts or set()

    mx = _prep(mx)
    h_base, k_max_web = build_h_base(mx, ios_h_base)

    cz = mx[
        (mx["step_k"] <= CLEAN_STEP_MAX)
        & (mx["weeks_obs"] >= mx["step_k"])
        & (~mx["app_id"].isin(crash_cohorts))
    ].copy()
    if cz.empty:
        raise RuntimeError("Clean zone is empty -- cannot fit lever
multipliers")
    cz["died"] = 1 - cz["survived"]
    cz["expected"] = h_base.reindex(cz["step_k"]).values

    multipliers, small_categories, support = {}, {}, {}
    for lever in levers:
        counts = cz[lever].value_counts()
        small = counts[counts < min_rows].index
        cz[lever] = cz[lever].where(~cz[lever].isin(small), "other")
        small_categories[lever] = small.tolist()

        g = cz.groupby(lever).agg(obs=("died", "mean"), exp=("expected",
"mean"), n=("died", "size"))
        m = (g["obs"] / g["exp"]).clip(*MAP_CLIP)
        multipliers[lever] = m.to_dict()
        multipliers[lever]["__default__"] = 1.0
        support[lever] = g

        cz["expected"] = cz["expected"] * cz[lever].map(m).fillna(1.0)

    return {
        "h_base": h_base,
        "k_max_web": k_max_web,
        "multipliers": multipliers,
        "small_categories": small_categories,
        "support": support,
        "levers": levers,
        "crash_cohorts": crash_cohorts,
    }

def sanity_check(state: dict, tol: tuple = SANITY_TOL) -> pd.DataFrame:
    rows = []
    for lever in state["levers"]:
        g = state["support"][lever]
        mult = pd.Series({k: v for k, v in
            state["multipliers"][lever].items() if k != "__default__"})
        common = g.index.intersection(mult.index)
        w_mean = float(np.average(mult.loc[common], weights=g.loc[common,
"n"]))
        rows.append({
            "lever": lever,
            "weighted_mean": w_mean,
            "ok": tol[0] <= w_mean <= tol[1],
            "categories": len(common),
            "collapsed to other": len(state["small_categories"][lever]),
            "rows": int(g["n"].sum()),

```

```

    })
    return pd.DataFrame(rows)

def _subs_with_levers(state: dict, rows: pd.DataFrame) -> pd.DataFrame:
    rows = _prep(rows)
    rows = _collapse_small(rows, state["small_categories"])
    return rows.sort_values("step_k").groupby("sub_id").first().reset_index()

def _combo_mult(state: dict, row) -> float:
    p = 1.0
    for lever in state["levers"]:
        table = state["multipliers"][lever]
        p *= table.get(row[lever], table["__default__"])
    return float(np.clip(p, *MAP_CLIP))

def personal_multiplier(state: dict, rows: pd.DataFrame) -> pd.DataFrame:
    subs = _subs_with_levers(state, rows)
    if not state["levers"]:
        return pd.DataFrame({"w": [len(subs)], "mult": [1.0]})
    combos = subs.groupby(state["levers"]).size().rename("w").reset_index()
    combos["mult"] = combos.apply(lambda r: _combo_mult(state, r), axis=1)
    return combos

def cohort_hr(state: dict, cohort_rows: pd.DataFrame, weeks: int) -> float:
    h_base = state["h_base"]
    combos = personal_multiplier(state, cohort_rows)
    subs = _subs_with_levers(state, cohort_rows)
    if state["levers"]:
        subs = subs.merge(combos[state["levers"] + ["mult"]],
                          on=state["levers"], how="left")
        sub_mult = subs.set_index("sub_id")["mult"]
    else:
        sub_mult = pd.Series(1.0, index=subs["sub_id"])

    obs = cohort_rows[cohort_rows["weeks_obs"] >= cohort_rows["step_k"] +
                      1].copy()
    obs = obs[obs["step_k"] <= weeks]
    n_seen = len(obs)
    if n_seen == 0:
        return 1.0

    obs["mult"] = obs["sub_id"].map(sub_mult)
    obs["exp_die"] = (h_base.reindex(obs["step_k"]).values *
                     obs["mult"].values)
    obs["exp_die"] = obs["exp_die"].clip(0.001, 0.999)
    g = obs.groupby("step_k").agg(o=("died", "mean"), e=("exp_die", "mean"),
                                n=("died", "size"))
    g = g[(g["n"] >= 20) & (g["e"] > 0)]
    log_hr = float(np.average(np.log(np.clip(g["o"] / g["e"], 0.2, 5.0)),
                              weights=g["n"])) if len(g) else 0.0
    return float(np.exp(log_hr * (n_seen / (n_seen + K_SHRINK))))

def cell_curve(state: dict, cell_rows: pd.DataFrame, hr: float) -> pd.Series:
    h_base = state["h_base"]
    combos = personal_multiplier(state, cell_rows)
    if combos.empty or combos["w"].sum() == 0:
        return pd.Series(np.nan, index=range(1, CURVE_MAX + 1))
    haz = np.clip(np.outer(combos["mult"].values, h_base.values) * hr, 0.001,
                  0.999)
    curves = np.cumprod(1 - haz, axis=1)
    w = combos["w"].values[:, None]
    return pd.Series((curves * w).sum(axis=0) / w.sum(), index=range(1,
                           CURVE_MAX + 1))

def predict(state: dict, rows: pd.DataFrame, weeks: int) -> pd.Series:
    return cell_curve(state, rows, cohort_hr(state, rows, weeks))

```

```

# ==== web/v3/ltv_v3/money.py ====
import numpy as np
import pandas as pd

from ltv_v3.config import (
    BASE_PRICE, UPS_PRICE, UPS_CADENCE_WEEKS, UPS_FIRST_WEEK, H_EXT,
)

def ups_checkpoints(upto: int, start: int = UPS_FIRST_WEEK, step: int =
    UPS_CADENCE_WEEKS):
    return list(range(start, upto + 1, step))

def _survival(curve: pd.Series, k: int) -> float:
    tail = float(curve.iloc[-1]) if len(curve) else 0.0
    return float(np.clip(curve.get(k, tail), 0.0, 1.0))

def project_increment(curve: pd.Series, anchor: int, horizon: int,
    share_base: float = 1.0, share_ups: float = 0.0) ->
    float:
    if horizon <= anchor:
        return 0.0
    total = 0.0
    checkpoints = set(ups_checkpoints(horizon))
    for k in range(anchor + 1, horizon + 1):
        alive = _survival(curve, k)
        total += share_base * BASE_PRICE * alive
        if k in checkpoints:
            total += share_ups * UPS_PRICE * alive
    return total

def cell_ltv(curve: pd.Series, observed_at_anchor: float, anchor: int,
    horizon: int,
    people: pd.DataFrame, share_ups: float = None) -> float:
    if people.empty:
        return observed_at_anchor
    has_base = people["has_base"].to_numpy() if "has_base" in people else
    np.ones(len(people), bool)
    share_base = float(has_base.mean())
    if share_ups is None:
        share_ups = float((people["has_ups"].to_numpy() & has_base).mean())
        if "has_ups" in people else 0.0
    return observed_at_anchor + project_increment(curve, anchor, horizon,
        share_base, share_ups)

def ltv_curve(curve: pd.Series, observed_at_anchor: float, anchor: int,
    people: pd.DataFrame, horizon: int = H_EXT) -> pd.Series:
    return pd.Series(
        {n: cell_ltv(curve, observed_at_anchor, anchor, n, people) for n in
        range(anchor, horizon + 1)}
    )

```